

Laboratory Report: Lab 3 - Inheritance and Polymorphism

Zofia Warzycha, 280971

March 2026

1 Task 1: Vehicle Hierarchy

1.1 Task Description

The primary objective of this task was to implement a vehicle management system utilizing inheritance and polymorphism.

Inheritance allows a child class to receive attributes and methods from a parent class, which avoids code duplication. I used an abstract class as a blueprint for all vehicles. An abstract class is a "concept" that cannot be instantiated directly; you cannot create a generic vehicle, only a specific one like a car or a truck.

Polymorphism was implemented to handle different objects through a single reference type, allowing the program to call overridden methods that behave differently depending on the specific object type.

1.2 Base Class: Vehicle

The `Vehicle` class serves as the parent. It defines common traits for every vehicle: brand, model, and maximum speed.

Listing 1: Abstract Vehicle class

```
1 public abstract class Vehicle {
2     private String brand;
3     private String model;
4     private double MaxSpeed;
5
6     public Vehicle(String brand, String model, double MaxSpeed) {
7         this.brand = brand;
8         this.model = model;
9         this.MaxSpeed = MaxSpeed;
10    }
11
12    public String getBrand() {
13        return brand;
14    }
15
16    public String getModel() {
17        return model;
18    }
19 }
```

```

20     public double getMaxSpeed() {
21         return MaxSpeed;
22     }
23
24     public void showInfo() {
25         System.out.println("Brand: " + getBrand() + ", model: " +
26             getModel() + ", max speed: "
27             + getMaxSpeed() + " km/h.");
28     }

```

1.3 Derived Classes: Car, Bike, and Truck

These classes "extend" the Vehicle. They use `super()` to let the parent handle the shared data, while they manage their unique attributes like doors, baskets, or load capacity.

Listing 2: Car class implementation

```

1 public class Car extends Vehicle {
2     private int numberOfDoors;
3
4     public Car(String brand, String model, double MaxSpeed, int
5         numberOfDoors) {
6         super(brand, model, MaxSpeed);
7         this.numberOfDoors = numberOfDoors;
8     }
9
10    public int getNumberOfDoors() {
11        return numberOfDoors;
12    }
13
14    @Override
15    public void showInfo() {
16        System.out.println("Car: " + getBrand() + ", model: " +
17            getModel() + ", max speed: "
18            + getMaxSpeed() + " km/h, with " + getNumberOfDoors() +
19            " doors.");
20    }
21
22    public void cityDriving() {
23        System.out.println(getBrand() + ", model: " + getModel() + " is
24            driving in the city.");
25    }
26 }

```

Listing 3: Bike class implementation

```

1 public class Bike extends Vehicle {
2     public boolean hasBasket;
3
4     public Bike(String brand, String model, double MaxSpeed, boolean
5         hasBasket) {
6         super(brand, model, MaxSpeed);
7         this.hasBasket = hasBasket;
8     }
9
10    public boolean getHasBasket() {

```

```

10         return hasBasket;
11     }
12
13     @Override
14     public void showInfo() {
15         System.out.println("Bike: " + getBrand() + ", model: " +
16             getModel() + ", max speed: " + getMaxSpeed() + " km/h, has
17             basket: " + getHasBasket());
18     }
19
20     public void ecoFriendly() {
21         System.out.println(getBrand() + ", model: " + getModel() + " is
22             eco friendly.");
23     }
24 }

```

Listing 4: Truck class implementation

```

1 public class Truck extends Vehicle {
2     public double loadCapacity;
3
4     public Truck(String brand, String model, double MaxSpeed, double
5         loadCapacity) {
6         super(brand, model, MaxSpeed);
7         this.loadCapacity = loadCapacity;
8     }
9
10    public double getLoadCapacity() {
11        return loadCapacity;
12    }
13
14    @Override
15    public void showInfo() {
16        System.out.println("Truck: " + getBrand() + ", model: " +
17            getModel() + ", max speed: "
18            + getMaxSpeed() + " km/h, with load capacity: " +
19            getLoadCapacity());
20    }
21
22    public void cargoTransport() {
23        System.out.println(getBrand() + ", model: " + getModel() + " is
24            a cargo transport.");
25    }
26 }

```

1.4 System Logic: VehicleDemo

This class ties everything together. It uses an array of the parent type (`Vehicle[]`) to store all different objects, demonstrating how polymorphism works in a loop.

Listing 5: VehicleDemo class

```

1 public class VehicleDemo {
2     public static void main(String[] args) {
3         Vehicle[] vehicles = new Vehicle[6];
4         vehicles[0] = new Car("Fiat", "Punto", 170, 5);
5         vehicles[1] = new Car("BMW", "E46", 180, 3);

```

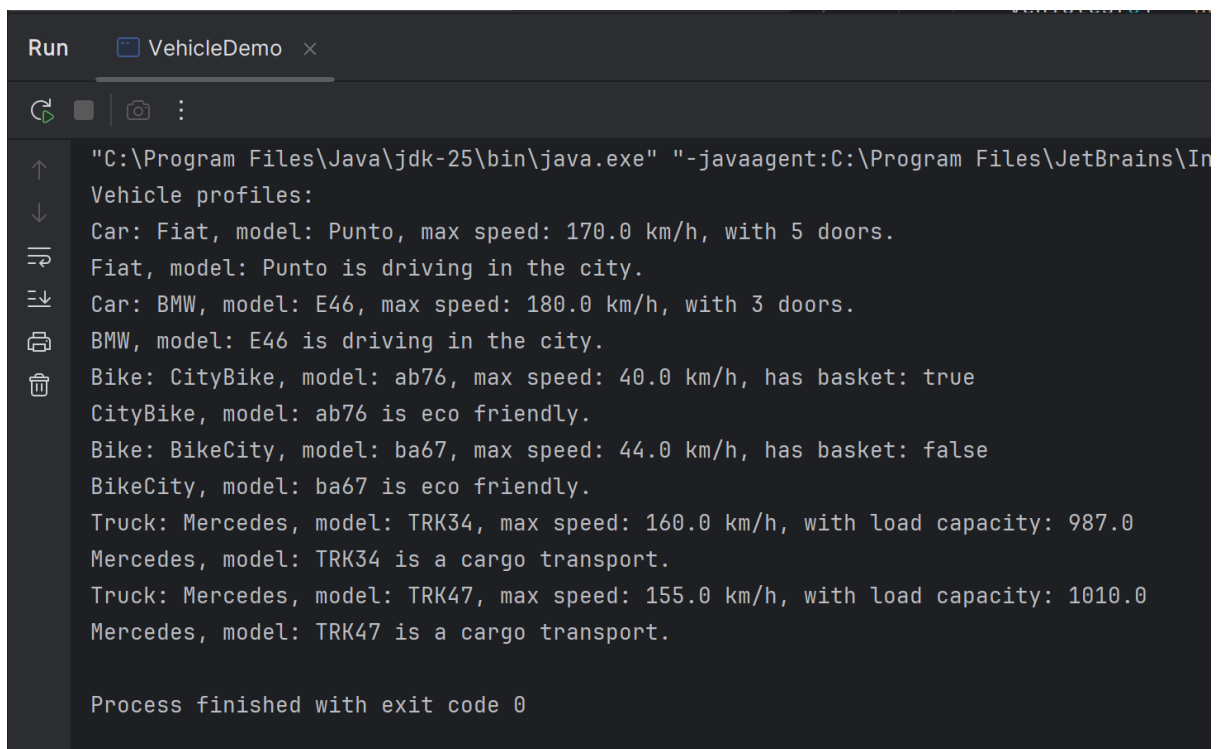
```

6      vehicles[2] = new Bike("CityBike", "ab76", 40, true);
7      vehicles[3] = new Bike("BikeCity", "ba67", 44, false);
8      vehicles[4] = new Truck("Mercedes", "TRK34", 160, 987);
9      vehicles[5] = new Truck("Mercedes", "TRK47", 155, 1010);
10
11     System.out.println("Vehicle profiles:");
12     for (int i = 0; i < vehicles.length; i++) {
13         if (vehicles[i] != null) {
14             vehicles[i].showInfo();
15             if (vehicles[i] instanceof Car) {
16                 Car myCar = (Car) vehicles[i];
17                 myCar.cityDriving();
18             } else if (vehicles[i] instanceof Bike) {
19                 Bike myBike = (Bike) vehicles[i];
20                 myBike.ecoFriendly();
21             } else if (vehicles[i] instanceof Truck) {
22                 Truck myTruck = (Truck) vehicles[i];
23                 myTruck.cargoTransport();
24             }
25         }
26     }
27 }
28 }

```

1.5 Console Output

Example output produced by the VehicleDemo program:



```

Run VehicleDemo x
C:\Program Files\Java\jdk-25\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\lib\idea_rt.jar=1273.0:C:\Program Files\Java\jdk-25\bin" -Dfile.encoding=UTF-8
Vehicle profiles:
Car: Fiat, model: Punto, max speed: 170.0 km/h, with 5 doors.
Fiat, model: Punto is driving in the city.
Car: BMW, model: E46, max speed: 180.0 km/h, with 3 doors.
BMW, model: E46 is driving in the city.
Bike: CityBike, model: ab76, max speed: 40.0 km/h, has basket: true
CityBike, model: ab76 is eco friendly.
Bike: BikeCity, model: ba67, max speed: 44.0 km/h, has basket: false
BikeCity, model: ba67 is eco friendly.
Truck: Mercedes, model: TRK34, max speed: 160.0 km/h, with load capacity: 987.0
Mercedes, model: TRK34 is a cargo transport.
Truck: Mercedes, model: TRK47, max speed: 155.0 km/h, with load capacity: 1010.0
Mercedes, model: TRK47 is a cargo transport.

Process finished with exit code 0

```

Figure 1: Console output

2 Task 2: Space Station Personnel Expansion

2.1 Task Description

In the second task, I applied inheritance to the Space Station project to model a diverse workforce. Instead of using one monolithic class, I created specialized roles. Each role inherits basic identification data from a parent class (`CrewMember` or `Officer`) but implements specific actions related to their profession, such as repairing the ship or issuing orders.

2.2 Crew Specialization

The `Engineer` and `Pilot` classes are prime examples of specialization. They call the parent constructor to handle shared data like name and age, but add unique attributes like a favorite tool or flight hours.

Listing 6: Engineer and Pilot implementations

```
1 public class Engineer extends CrewMember {
2     private String favoriteTool;
3
4     public Engineer(String name, int age, String specialization, double
5         missionHours, String favoriteTool) {
6         super(name, age, Role.ENGINEER, specialization, missionHours);
7         this.favoriteTool = favoriteTool;
8     }
9
10    public String getFavoriteTool() { return favoriteTool; }
11
12    public void repairShip() {
13        System.out.println(getName() + " is repairing the ship using a
14            " + favoriteTool + "!");
15    }
16 }
17
18 public class Pilot extends CrewMember {
19     private int flightHours;
20
21    public Pilot(String name, int age, String specialization, double
22        missionHours, int flightHours) {
23        super(name, age, Role.PILOT, specialization, missionHours);
24        this.flightHours = flightHours;
25    }
26
27    public int getFlightHours() { return flightHours; }
28
29    public void flyShip() {
30        System.out.println(getName() + " is flying the ship! They have
31            " + flightHours + " hours of flight experience.");
32    }
33 }
```

2.3 Officer Specialization

The Officer hierarchy demonstrates a different chain of command. A `CommandOfficer` has methods for issuing orders, while a `MedicalOfficer` focuses on healing crew members.

Listing 7: `CommandOfficer` and `MedicalOfficer` implementations

```
1 public class CommandOfficer extends Officer {
2     private int securityClearanceLevel;
3
4     public CommandOfficer(String name, String rank, String department,
5         int yearsOfExperience, int securityClearanceLevel) {
6         super(name, rank, department, yearsOfExperience);
7         this.securityClearanceLevel = securityClearanceLevel;
8     }
9
10    public int getSecurityClearanceLevel() { return
11        securityClearanceLevel; }
12
13    public void issueOrder(String order) {
14        System.out.println(getRank() + " " + getName() + " issues an
15            order: " + order);
16    }
17 }
18
19 public class MedicalOfficer extends Officer {
20     private String medicalSpecialty;
21
22     public MedicalOfficer(String name, String rank, String department,
23         int yearsOfExperience, String medicalSpecialty) {
24         super(name, rank, department, yearsOfExperience);
25         this.medicalSpecialty = medicalSpecialty;
26     }
27
28    public String getMedicalSpecialty() { return medicalSpecialty; }
29
30    public void healCrewMember(String patientName) {
31        System.out.println("Dr. " + getName() + " is treating " +
32            patientName + " using their background in " +
33            medicalSpecialty + ".");
34    }
35 }
```

2.4 Access Card Hierarchy

Inheritance is also useful for administrative systems. The `PremiumAccessCard` inherits from the basic `AccessCard` but adds an expiration date. It uses `super.toString()` to retrieve the base information and concatenates the premium features.

Listing 8: Access Card System

```
1 import java.time.LocalDate;
2
3 public class AccessCard {
4     private String accessLevel;
5     private String crewMemberName;
6     private LocalDate issueDate;
7 }
```

```

8      public AccessCard(String accessLevel, String crewMemberName) {
9          this.accessLevel = accessLevel;
10         this.crewMemberName = crewMemberName;
11         this.issueDate = LocalDate.now();
12     }
13
14     public String getAccessLevel() { return accessLevel; }
15     public String getCrewMemberName() { return crewMemberName; }
16     public LocalDate getIssueDate() { return issueDate; }
17
18     @Override
19     public String toString() {
20         return "Access Card of " + crewMemberName + ", level: " +
21             accessLevel + ", issued: " + issueDate + "];";
22     }
23 }
24 public class PremiumAccessCard extends AccessCard {
25     private LocalDate expirationDate;
26     private String specialPrivileges;
27
28     public PremiumAccessCard(String accessLevel, String crewMemberName,
29         String specialPrivileges) {
30         super(accessLevel, crewMemberName);
31         this.specialPrivileges = specialPrivileges;
32         this.expirationDate = LocalDate.now().plusYears(1);
33     }
34
35     @Override
36     public String toString() {
37         return super.toString() + " -> PREMIUM [Expires: " +
38             expirationDate + ", privileges: " + specialPrivileges + "];";
39     }
40 }

```

2.5 System Tester

The tester class brings all the specialized classes together, proving that the inheritance logic allows for smooth object creation and method execution.

Listing 9: Space Station Tester Class

```

1 public class List3_tester {
2     public static void main(String[] args) {
3         Engineer chiefEngineer = new Engineer("Zofia Warzycha", 35, "
4             ship maintenance", 1500.5, "axe");
5         CommandOfficer captain = new CommandOfficer("Michal Michalowski
6             ", "captain", "command", 25, 10);
7
8         System.out.println("\n-----Personnel info:");
9         chiefEngineer.printProfile();
10        chiefEngineer.repairShip();
11        System.out.println();
12        captain.issueOrder("Prepare the ship for war.");
13
14        System.out.println("\n-----System of access cards:");
15    }
16 }

```

```

13     AccessCard basicCard = new AccessCard("level 2", chiefEngineer.
        getName());
14     PremiumAccessCard vipCard = new PremiumAccessCard("level 10",
        captain.getName(), "AllInclusive service");
15
16     System.out.println(basicCard);
17     System.out.println(vipCard);
18 }
19 }

```

2.6 Console Output

Example output produced by the `List3_tester` program:

```

Run List3_tester x
"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.2.3\lib\idea_rt.jar=53299" -Dfile.encoding=UTF-8
-----Personnel info:
Hello! Meet Zofia Warzycha, a 35-year-old ENGINEER. This crew member specializes in ship maintenance and has already completed 1500.5 mission hours.
Zofia Warzycha is repairing the ship using a axe!

captain Michał Michałowski issues an order: Prepare the ship for war.

-----System of access cards:
Access Card of Zofia Warzycha, level: Level 2, issued: 2026-03-25]
Access Card of Michał Michałowski, level: Level 10, issued: 2026-03-25] -> PREMIUM [Expires: 2027-03-25, privileges: AllInclusive service]

Process finished with exit code 0

```

Figure 2: Console output

3 Conclusions

- I learned that inheritance (**extends**) is a great way to save time. Instead of copying and pasting the same code for every class, I can just write it once in the parent class.
- I noticed that making a class **abstract** stops me from accidentally creating a blank, generic object (like a "Vehicle" that isn't a car or a bike).
- I figured out that I have to use the **super()** keyword as the very first line in a child's constructor. It acts like a phone call to the parent class to set up the basic data first.
- I did an experiment with an array of Vehicles and saw Polymorphism in action. It's really cool that Java knows exactly which **showInfo()** method to use for each object. I also learned to use **instanceof** to safely check what an object actually is before using its special methods.